

KatzLab Ureteroscope

ROS2 Bridge, Isaac Sim Digital Twin, and the Reach-Task Runbook

Step-by-step setup and run guide

October 5, 2026

How to use this document

This is a linear runbook: each section assumes the previous one actually worked. If a step fails, stop and fix it before moving to the next section — later steps depend on earlier ones being genuinely up and running, not just “probably fine.”

Two machines are involved:

- **Mint box** — the machine physically wired to the rig (Teensy, motors, laser), running ROS2 and Isaac Sim.
- **Mac** — your laptop, used only to *watch* what the Mint box is doing. Nothing on the Mac drives the rig.

Contents

1	Install Foxglove Studio (on the Mac)	3
2	Install ROS2 (on the Mint box)	3
2.1	Check the Mint/Ubuntu base	3
2.2	Run the setup script	3
2.3	Open a new terminal and verify	3
3	Build and run the bridge (on the Mint box)	4
3.1	Sanity-check the parts that do not need ROS2 at all	4
3.2	Build the ROS2 package	4
3.3	Start the bridge, with Foxglove watching	4
4	Watch it from the Mac	4
5	Try sending a motion command (on the Mint box)	5
6	Isaac Sim: look at the real rig before anything else	5
7	Isaac Sim: confirm the live mirror	5

8	First training milestone: get the tip to the stone	6
8.1	Smoke-test the environment	6
8.2	Install the training dependencies	6
8.3	Train	6
9	What to report back if something breaks	6

1 Install Foxglove Studio (on the Mac)

Foxglove Studio lets you watch the live ROS2 graph (topics, values, a 3D view) from the Mac **without installing ROS2 there at all**. It talks to a small bridge running on the Mint box over a plain WebSocket.

1. On the Mac, download and install **Foxglove Studio** (search for it by name — it is a free macOS app, no other dependency needed).
2. Open it once to confirm it launches. You will come back to it in Section 4, after the Mint-side bridge is running.

2 Install ROS2 (on the Mint box)

2.1 Check the Mint/Ubuntu base

The install script refuses to run on anything other than a Mint 22.x (Ubuntu 24.04 “noble”) base, on purpose — running the wrong ROS2 distro against the wrong base fails in confusing ways otherwise. Confirm first:

```
cat /etc/upstream-release/lsb-release
```

You want to see `DISTRIB_CODENAME=noble`. If it says something else, stop and ask before continuing — the script will bail out partway through otherwise.

2.2 Run the setup script

From the repository root on the Mint box:

```
cd katzlab-ureteroscope
./ros2/setup-mint.sh
```

This installs ROS2 Jazzy Desktop (includes RViz2), `colcon`, and `foxglove-bridge`. It is a few GB and will take a while. It is **safe to re-run** if it fails partway through.

2.3 Open a new terminal and verify

ROS2 is added to your shell’s startup file by the script, so a terminal that was already open **will not** see it. Open a brand new terminal, then:

```
ros2 --version
which colcon
```

Both commands should print something, not “command not found.” If either fails, ROS2 did not install cleanly — re-run `./ros2/setup-mint.sh` before continuing.

3 Build and run the bridge (on the Mint box)

3.1 Sanity-check the parts that do not need ROS2 at all

These run anywhere Python runs, right now, before touching ROS2 or Isaac Sim. If either fails, fix it first — nothing downstream will work:

```
cd ros2/katzlab_bridge/test && python3 -m pytest -q
cd ../../isaac/training && python3 -m pytest test_reach_math.py -q
```

Expected: 6 passed, 1 skipped and 10 passed.

3.2 Build the ROS2 package

```
cd ros2
colcon build --packages-select katzlab_bridge
source install/setup.bash
```

3.3 Start the bridge, with Foxglove watching

This starts the `katzlab_bridge` node *and* the `foxglove_bridge` WebSocket server together, so the Mac can watch everything from Section 4 onward:

```
ros2 launch katzlab_bridge watch.launch.py dry_run:=true
```

`dry_run:=true` means no Teensy/hardware needed for this step — it exercises the ROS2 wiring only. Drop the flag once hardware is attached and you want the real thing.

Leave this terminal running. Everything else happens in other terminals (or on the Mac).

4 Watch it from the Mac

1. Find the Mint box's IP address (on Mint):

```
hostname -I
```

2. On the Mac, open Foxglove Studio → **Open connection** → **Foxglove WebSocket** → enter:

```
ws://<mint-ip-address>:8765
```

3. You should immediately see topics appear, including `/ureteroscope/joint_states` and `/ureteroscope/laser`. Open one and confirm values are actually updating, not just that the topic exists.

If nothing appears: confirm both machines are on the same network/reachable from each other, and that the `watch.launch.py` terminal on Mint is still running without errors.

5 Try sending a motion command (on the Mint box)

From a *second* terminal on Mint (the first stays running the bridge):

```
source ~/katzlab-ureteroscope/ros2/install/setup.bash

ros2 topic pub /ureteroscope/cmd_velocity geometry_msgs/msg/Twist \
  "{linear: {x: 0.001}, angular: {z: 0.0, y: 0.0}}"
```

You should see `/ureteroscope/joint_states` values change, both in a terminal `ros2 topic echo` and in Foxglove on the Mac.

Other useful checks from this second terminal:

```
ros2 service call /ureteroscope/home std_srvs/srv/Trigger
ros2 service call /ureteroscope/estop std_srvs/srv/Trigger
```

6 Isaac Sim: look at the real rig before anything else

This step needs a GUI window, so it has to run directly on the Mint box's own screen (or via VNC/remote desktop into it) — Foxglove cannot show you this part.

```
source /opt/ros/jazzy/setup.bash
~/isaacsim/python.sh ~/katzlab-ureteroscope/ros2/isaac/preview_scene.py
```

A window opens with the real rig CAD (linear stage, rotation bearing, the actual ureteroscope mesh) and the real 8-stone benchtop target grid. Check, before moving on to anything else:

1. Does the scope's length look visually right relative to the table/linear stage? (There is a known open question about this — see `ros2/isaac/assets/ASSET-NOTES.md` in the repo.)
2. Do the 7 flexible-tip segments bend plausibly as a continuous curve when driven through their range, rather than kinking at one joint?
3. Do the 8 stone markers sit somewhere sensible relative to where the tip points?

If something looks visually wrong, stop here and report back what you see before training anything against it.

7 Isaac Sim: confirm the live mirror

With the bridge from Section 4 still running:

```
~/isaacsim/python.sh ~/katzlab-ureteroscope/ros2/isaac/load_rig.py
```

Re-send the test command from Section 5. The rig in the Isaac Sim window should move within a step or two of the real/dry-run joint states updating.

8 First training milestone: get the tip to the stone

8.1 Smoke-test the environment

No training yet — just confirms the simulated environment runs end to end with random actions:

```
~/isaacsim/python.sh \  
~/katzlab-ureteroscope/ros2/isaac/training/reach_task.py
```

8.2 Install the training dependencies

```
~/isaacsim/python.sh -m pip install gymnasium numpy stable-baselines3
```

8.3 Train

```
~/isaacsim/python.sh \  
~/katzlab-ureteroscope/ros2/isaac/training/train_reach.py \  
--total-timesteps 200000
```

This is state-based (tip and target *positions*, not camera images) on purpose — it is the simpler problem, and has to work before a vision-based version is worth attempting. Do not expect reliable stone-reaching from one run; watch for the mean episode reward trending up and the mean episode length trending down as the first sign that anything is working at all.

9 What to report back if something breaks

For each step above, the most useful thing to send back is:

- Which numbered section/step you were on.
- The **exact** command you ran.
- The **full** error output, not a paraphrase.

Sections 7–9 (the Isaac Sim pieces) are the ones most likely to need a fix on first contact with a real Isaac Sim install — none of that code has been run against one yet, and the two most likely failure points (the URDF importer extension’s exact module path, and Replicator/RL API field names) are both named in `ros2/isaac/README.md`.